

main

February 9, 2026

```
[1]: import numpy as np
import pandas as pd

[2]: train_data=pd.read_csv(r'data/train.csv')
test_data=pd.read_csv(r'data/test.csv')

[3]: full_data=[train_data,test_data]

[4]: import re
def get_title(name):
    title_search = re.search(r' ([A-Za-z]+\.)\.', name)
    if title_search:
        return title_search.group(1)
    return ""

for df in full_data:
    df['Title'] = df['Name'].apply(get_title)
    df['Title'] = df['Title'].replace(['Lady', 'Countess','Capt', 'Col', 'Don', 'Dr', 'Major', 'Rev', 'Sir', 'Jonkheer', 'Dona'], 'Rare')
    df['Title'] = df['Title'].replace('Mlle', 'Miss')
    df['Title'] = df['Title'].replace('Ms', 'Miss')
    df['Title'] = df['Title'].replace('Mme', 'Mrs')

[5]: for df in full_data:
    df["Embarked"]=df["Embarked"].fillna(train_data["Embarked"].mode()[0])
    df["Fare"]=df["Fare"].fillna(train_data["Fare"].median())
    df['Age'] = df['Age'].fillna(df.groupby('Title')['Age'].transform('median'))

[6]: X_classes=['Pclass','Title','Sex','Age','SibSp','Parch','Fare','Embarked']
need_onehot=(1,2,7)
X_train_uncodded=train_data[X_classes].to_numpy()
X_test_uncodded=test_data[X_classes].to_numpy()
X_all_uncodded=np.concat([X_train_uncodded,X_test_uncodded])
y_train=train_data['Survived'].to_numpy()

[7]: from sklearn.preprocessing import OneHotEncoder

encoder=OneHotEncoder()
```

```

encoder.fit(X_all_uncodedd[:,need_onehot])
one_hotted=encoder.transform(X_train_uncodedd[:,need_onehot]).toarray()
X_train=np.concatenate((np.
    ↳delete(X_train_uncodedd,need_onehot,axis=1),one_hotted),axis=1)
one_hotted=encoder.transform(X_test_uncodedd[:,need_onehot]).toarray()
X_test=np.concatenate((np.
    ↳delete(X_test_uncodedd,need_onehot,axis=1),one_hotted),axis=1)

```

```

[8]: from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.
    ↳2, random_state=42)

```

```

[9]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

num_trees=[100,150,200,500]
depths=range(3,20,2)
best_rf=-1
best_model_rf=None
best_params_rf=None

for num_tree in num_trees:
    for depth in depths:
        clf=RandomForestClassifier(n_estimators=num_tree,max_depth=depth)
        clf.fit(X_train,y_train)
        y_pred_val=clf.predict(X_val)
        y_pred_train=clf.predict(X_train)
        acc_train=accuracy_score(y_pred_train,y_train)
        acc_val=accuracy_score(y_pred_val,y_val)
        print((num_tree,depth,acc_train,acc_val))
        if acc_val>best_rf:
            best_rf=acc_val
            best_model_rf=clf
            best_params_rf=(num_tree,depth)

print(best_params_rf,best_rf)

```

```

(100, 3, 0.8174157303370787, 0.8044692737430168)
(100, 5, 0.8497191011235955, 0.8156424581005587)
(100, 7, 0.8876404494382022, 0.8212290502793296)
(100, 9, 0.9213483146067416, 0.8100558659217877)
(100, 11, 0.9480337078651685, 0.8212290502793296)
(100, 13, 0.9606741573033708, 0.8324022346368715)
(100, 15, 0.9676966292134831, 0.8212290502793296)
(100, 17, 0.9789325842696629, 0.8324022346368715)
(100, 19, 0.9775280898876404, 0.8324022346368715)
(150, 3, 0.8174157303370787, 0.8044692737430168)

```

```
(150, 5, 0.8623595505617978, 0.8212290502793296)
(150, 7, 0.8848314606741573, 0.8156424581005587)
(150, 9, 0.9171348314606742, 0.8156424581005587)
(150, 11, 0.9536516853932584, 0.8435754189944135)
(150, 13, 0.9620786516853933, 0.8212290502793296)
(150, 15, 0.9705056179775281, 0.8324022346368715)
(150, 17, 0.9775280898876404, 0.8212290502793296)
(150, 19, 0.9803370786516854, 0.8268156424581006)
(200, 3, 0.8174157303370787, 0.8044692737430168)
(200, 5, 0.8567415730337079, 0.8100558659217877)
(200, 7, 0.8932584269662921, 0.8156424581005587)
(200, 9, 0.9311797752808989, 0.8268156424581006)
(200, 11, 0.949438202247191, 0.8435754189944135)
(200, 13, 0.9620786516853933, 0.8156424581005587)
(200, 15, 0.9705056179775281, 0.8268156424581006)
(200, 17, 0.9775280898876404, 0.8268156424581006)
(200, 19, 0.9803370786516854, 0.8212290502793296)
(500, 3, 0.8174157303370787, 0.8044692737430168)
(500, 5, 0.8553370786516854, 0.8156424581005587)
(500, 7, 0.8918539325842697, 0.8156424581005587)
(500, 9, 0.9269662921348315, 0.8156424581005587)
(500, 11, 0.952247191011236, 0.8379888268156425)
(500, 13, 0.9634831460674157, 0.8268156424581006)
(500, 15, 0.9691011235955056, 0.8268156424581006)
(500, 17, 0.9775280898876404, 0.8324022346368715)
(500, 19, 0.9803370786516854, 0.8156424581005587)
(150, 11) 0.8435754189944135
```

```
[10]: from sklearn.preprocessing import StandardScaler
```

```
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_val_scaled=scaler.transform(X_val)
X_test_scaled=scaler.transform(X_test)
```

```
[11]: from sklearn.linear_model import LogisticRegression
```

```
Cs=[1e-2,1e-1,1,1e1,1e2,1e3]
penalties=['l1','l2']
best_lr=-1
best_model_lr=None
best_params_lr=None

for C in Cs:
    for penalty in penalties:
        model_lr=LogisticRegression(C=C,solver='liblinear',penalty=penalty)
        model_lr.fit(X_train_scaled,y_train)
```

```

y_pred_val=model_lr.predict(X_val_scaled)
y_pred_train=model_lr.predict(X_train_scaled)
acc_train=accuracy_score(y_pred_train,y_train)
acc_val=accuracy_score(y_pred_val,y_val)
print((C,penalty,acc_train,acc_val))
if acc_val>best_lr:
    best_lr=acc_val
    best_model_lr=model_lr
    best_params_lr=(C,penalty)

print(best_params_lr,best_lr)

```

```

(0.01, 'l1', 0.7823033707865169, 0.7821229050279329)
(0.01, 'l2', 0.8132022471910112, 0.8100558659217877)
(0.1, 'l1', 0.8342696629213483, 0.8212290502793296)
(0.1, 'l2', 0.8356741573033708, 0.8156424581005587)
(1, 'l1', 0.8328651685393258, 0.8100558659217877)
(1, 'l2', 0.8328651685393258, 0.8100558659217877)
(10.0, 'l1', 0.8342696629213483, 0.8100558659217877)
(10.0, 'l2', 0.8342696629213483, 0.8100558659217877)
(100.0, 'l1', 0.8342696629213483, 0.8100558659217877)
(100.0, 'l2', 0.8342696629213483, 0.8100558659217877)
(1000.0, 'l1', 0.8342696629213483, 0.8100558659217877)
(1000.0, 'l2', 0.8342696629213483, 0.8100558659217877)
(0.1, 'l1') 0.8212290502793296

```

```

/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.

```

```

warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.

```

```

warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.

```

```

warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,

```

```

leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-

```

```

packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1160: UserWarning: Inconsistent
values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio
only.
    warnings.warn(
/home/x1879/miniconda3/envs/cs231n/lib/python3.12/site-
packages/sklearn/linear_model/_logistic.py:1135: FutureWarning: 'penalty' was
deprecated in version 1.8 and will be removed in 1.10. To avoid this warning,
leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use
l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and
C=np.inf instead of penalty=None.
    warnings.warn(

```

```
[12]: from sklearn.neighbors import KNeighborsClassifier
```

```

ns=range(1,16,2)

best_knn=-1
best_model_knn=None
best_params_knn=None
for n in ns:
    knn=KNeighborsClassifier(n_neighbors=n)
    knn.fit(X_train_scaled,y_train)
    y_pred_val=knn.predict(X_val_scaled)
    y_pred_train=knn.predict(X_train_scaled)
    acc_train=accuracy_score(y_pred_train,y_train)
    acc_val=accuracy_score(y_pred_val,y_val)
    print((n,acc_train,acc_val))
    if acc_val>best_knn:
        best_knn=acc_val
        best_model_knn=knn
        best_params_knn=(n)
print(best_params_knn,best_knn)

```

```

(1, 0.976123595505618, 0.7597765363128491)
(3, 0.8693820224719101, 0.8100558659217877)
(5, 0.8651685393258427, 0.8156424581005587)
(7, 0.8539325842696629, 0.8268156424581006)
(9, 0.8398876404494382, 0.8212290502793296)
(11, 0.8412921348314607, 0.8156424581005587)
(13, 0.8342696629213483, 0.7988826815642458)
(15, 0.8314606741573034, 0.8044692737430168)
7 0.8268156424581006

```

```

[13]: def generate_submission(model,name,X):
        y_pred=model.predict(X)
        submission = pd.DataFrame({
            'PassengerId': test_data['PassengerId'],
            'Survived': y_pred
        })
        submission.to_csv(f'sub_{name}.csv',index=False)
        return y_pred

```

```

[14]: models=[best_model_rf,best_model_lr,best_model_knn]
        names=['rf','lr','knn']
        Xs=[X_test,X_test_scaled,X_test_scaled]
        ys=[]
        for model,name,X in zip(models,names,Xs):
            ys.append(generate_submission(model,name,X))

```

```

[15]: y_all=np.where(ys[0]+ys[1]+ys[2]>0.5,1,0)
        submission = pd.DataFrame({
            'PassengerId': test_data['PassengerId'],

```

```
'Survived': y_all  
})  
submission.to_csv(f'sub_all.csv', index=False)
```